

Software Design Description - Milestone 2

1. Scopul Designului

Acest document descrie designul preliminar al aplicatiei SQL Code Generator si il coreleaza cu implementarea existenta. Designul acopera componentele C si Python, protocolul binar, coada FIFO, job tracking-ul, validarea SQL, administrarea serverului si generarea documentatiei PDF.

Implementarea analizata se afla in `src/`, `include/`, `clients/`, `examples/`, `Makefile` si `.clang-tidy`. Documentul pastreaza legatura intre deciziile de proiectare si fisierele concrete din repository.

2. Arhitectura Generala

Sistemul este organizat in componente separate pentru server, client ordinar, client admin, client Python, model de date si protocol. Serverul este procesul TCP principal si expune doua porturi: unul pentru clientii ordinari IN si unul pentru administrare UX. Clientul ordinar C foloseste serviciile de upload schema, generare SQL, download SQL si validare INSERT. Clientul admin C/ncurses produce rapoarte si executa operatii de mentenanta. Clientul Python demonstreaza interoperabilitatea protocolului binar. Modulul `model` concentreaza schema ER, generarea SQL, validarea si rapoartele, iar modulul `protocol` concentreaza headerul binar si functiile de citire/scriere completa pe socket.

Fluxul principal incepe cu pornirea serverului, citirea configuratiei si deschiderea celor doua socketuri de ascultare. Thread-ul IN accepta clienti ordinari si le citeste mesajele prin `poll()`. Operatiile ordinare care necesita procesare sunt impachetate in `Request` si introduse in `RequestQueue`. Workerul FIFO proceseaza cererile in ordinea primirii si actualizeaza `SharedState`. Thread-ul admin raspunde sincron la rapoarte si comenzi administrative. Clientii ordinari pot interoga ulterior starea si rezultatul joburilor prin `OP_JOB_STATUS` si `OP_JOB_RESULT`.

3. Protocol

Protocolul este implementat in `include/protocol.h` si `src/protocol.c`. Fiecare mesaj foloseste structura `MsgHeader`, cu campurile `msg_size`, `client_id`, `op_id` si `flags`. Campurile sunt serializate in network byte order, iar payload-ul binar are exact dimensiunea indicata de `msg_size`.

Modulul de protocol ofera conectare TCP prin `connect_tcp`, socket de ascultare prin `listen_tcp`, citire si scriere completa prin `read_full` si `write_full`, precum si functii pentru trimiteria si receptionarea mesajelor complete. Dimensiunea maxima a payloadului este limitata prin `SQLCG_MAX_PAYLOAD`, iar transferurile de fisiere folosesc bucati de 64 KB definite prin `SQLCG_FILE_CHUNK`.

Operatiile definite in protocol acopera sesiunea ordinara, uploadul schemei, downloadul SQL generat, generarea SQL, validarea INSERT, interogarea joburilor, raspunsurile `OP_OK` si `OP_ERROR`, autentificarea admin, rapoartele admin si operatiile active de deconectare, anulare si blocare.

4. Modelul de Date

Modelul de date este definit in `include/model.h` si implementat in `src/model.c`. Structurile principale sunt `Column`, `Row`, `Table`, `ClientInfo`, `JobInfo` si `SharedState`. `SharedState` retine statisticile

globale, clientii activi, joburile recente, ultimul job al fiecarui client, comanda activa, cererea de anulare, schema ER incarcata, randurile acceptate si istoricul circular pentru rapoarte.

Functia `state_init` initializeaza starea comuna. Functia `load_er_json` parseaza schema ER cu cJSON si completeaza tabelele interne. Functia `generate_sql` construiește DDL-ul `CREATE TABLE`. Functiile `validate_insert_batch` si `apply_insert_batch` valideaza batch-uri `INSERT`, prima fara modificarea starii, a doua cu aplicarea randurilor acceptate. Functia `admin_report` construiește rapoartele folosite de clientul de administrare.

5. Server

Serverul este implementat in `src/server.c`. La pornire, acesta incarca valorile runtime in `RuntimeConfig`, creeaza directoarele `data/` si `logs/`, deschide `logs/server.log`, alocă `SharedState` prin `mmap(MAP_SHARED | MAP_ANONYMOUS)`, initializeaza `RequestQueue`, deschide socketurile pentru clienti ordinari si admin, apoi porneste trei fire de executie: workerul FIFO, interfata ordinara si interfata admin.

Thread-ul ordinar foloseste `poll()` pentru socketul de ascultare si pentru clientii activi. La acceptarea unei conexiuni, serverul verifica blocklist-ul, memoreaza adresa IP si asteapta handshake-ul `OP_CONNECT`. Dupa conectare, clientul primeste un `client_id`. Operatiile `OP_BYE`, `OP_JOB_STATUS` si `OP_JOB_RESULT` sunt tratate direct, iar restul operatiilor sunt puse in coada FIFO ca obiecte `Request`.

Thread-ul worker extrage cereri din coada cu `pthread_cond_wait`, le proceseaza in ordinea primirii si actualizeaza joburile din `QUEUED` in `RUNNING`, apoi in `DONE` sau `ERROR`. Tot aici sunt actualizate statisticile administrative, istoricul si rezultatul jobului. Raspunsul este trimis inapoi catre socketul clientului ordinar.

Thread-ul admin accepta un singur client de administrare la un moment dat. Daca adminul ramane inactiv peste timeoutul configurat, conexiunea este inchisa si serverul permite conectarea altui admin. Mesajele admin sunt procesate sincron in `handle_admin_message`, folosind aceeasi stare comuna.

6. Clienti

Clientul ordinar C este implementat in `src/client.c` si suporta atat mod interactiv REPL, cat si mod neinteractiv pentru demo. Comenzile disponibile permit upload de schema, generare SQL, download de SQL, validare prin `insert` sau `insertfile`, interogare `status`, interogare `result` si inchidere prin `quit`.

Clientul admin este implementat in `src/admin_client.c`. In modul ncurses, acesta permite selectarea rapoartelor si operatiilor prin taste. In modul neinteractiv, aceleasi actiuni sunt disponibile prin argumente CLI, ceea ce permite demonstrarea automata a rapoartelor `clients`, `commands`, `avg`, `history`, `tables`, `queue` si `jobs`, precum si a operatiilor `-disconnect`, `-cancel` si `-block`.

Clientul Python din `clients/python_client.py` implementeaza acelasi protocol binar pentru clientul ordinar. El confirma ca protocolul este independent de implementarea C a clientului si ca serverul nu depinde de scripturi externe pentru procesarea datelor.

7. Transfer de Fisiere

Uploadul schemei ER incepe cu `OP_UPLOAD_BEGIN`, continua cu mai multe mesaje `OP_UPLOAD_CHUNK` si se finalizeaza cu `OP_UPLOAD_END`. Serverul scrie datele in

`data/upload_<client_id>.schema.json`, apoi apeleaza `load_er_json`. Fiecare chunk are cel mult 64 KB, iar fisierul temporar este separat pe client.

Downloadul SQL generat incepe cand clientul trimite `OP_DOWNLOAD_SQL`. Serverul genereaza SQL-ul cu `generate_sql`, il scrie in `data/generated_<client_id>.sql`, trimite `OP_DOWNLOAD_BEGIN` cu nume si dimensiune, apoi transmite continutul prin `OP_DOWNLOAD_CHUNK` si incheie transferul cu `OP_DOWNLOAD_END`. Aceasta solutie demonstreaza transfer bidirectional si evita dependenta de un singur payload mare.

8. Validare SQL si Aplicare Date

Validarea `INSERT` are doua straturi. Primul strat foloseste `libpg_query` pentru analiza sintactica PostgreSQL. Al doilea strat este validatorul semantic intern, care verifica existenta tabeli, existenta coloanelor, potrivirea numarului de valori, constrangerile `NOT NULL`, unicitatea pentru `PRIMARY KEY` si `UNIQUE`, duplicatele aparute in acelasi batch si existenta randurilor referite prin foreign key.

Pentru fiecare batch, serverul creeaza un proces copil cu `fork()`. Copilul apeleaza `validate_insert_batch` pe starea comuna mapata prin `mmap` si trimite rezultatul prin pipe catre parinte. Parintele asteapta copilul cu `waitpid`; daca validarea a reusit, parintele apeleaza `apply_insert_batch` si actualizeaza randurile memorate. Prin aceasta separare, validarea este izolata, iar aplicarea datelor ramane controlata de procesul serverului.

9. Administrare

Rapoartele admin sunt produse de `admin_report`. Raportul `clients` afiseaza clientii ordinari conectati, descriptorul, adresa si timpul de inactivitate. Raportul `commands` afiseaza totalul comenzilor, numarul de inserturi, esecurile, anularile si comanda activa. Raportul `avg` afiseaza durata medie de executie, `history` afiseaza istoricul circular, `tables` afiseaza tabelele si numarul de randuri, `queue` afiseaza adancimea cozii FIFO si anulara pending, iar `jobs` afiseaza joburile recente si statusul lor.

Operatiile active sunt implementate in `handle_admin_message`. Pentru deconectare, serverul cauta socketul clientului, ii trimite notificare si inchide conexiunea cu `shutdown`. Pentru anulare, serverul seteaza `cancel_client_id`, iar workerul respinge cererea observabila pentru acel client. Pentru blocare, serverul adauga adresa in blacklist, iar conexiunile IN viitoare de la acea adresa sunt refuzate la accept.

10. Configurare si Logging

Configurarea runtime este incarcata in `cfg_load`. Serverul porneste de la valori implicite, apoi citește `config.cfg`, aplica variabilele de mediu si in final aplica argumentele CLI. Daca `libconfig` este disponibil, sunt citite campurile `server.port`, `server.admin_port` si `server.admin_timeout`. Daca biblioteca nu este disponibila, exista un fallback simplu pentru linii `cheie = valoare`.

Logging-ul este implementat prin `log_event` si scrie in `logs/server.log`. Accesul la fisierul de log este protejat de `g_log_mutex`. Logul contine evenimente precum pornirea serverului, acceptarea clientilor, crearea joburilor, schimbarea statusurilor si oprirea serverului.

11. Concurenta si Sincronizare

Designul foloseste `pthread_create` pentru cele trei fire de executie, `poll()` pentru multiplexarea conexiunilor pe interfetele IN si UX, `pthread_mutex_t` si `pthread_cond_t` pentru coada FIFO, `mmap` cu `MAP_SHARED` pentru starea consultata de procesul copil si pipe anonim pentru comunicarea copil-parinte la validarea INSERT.

Modificarile principale ale starii sunt centralizate in worker, iar adminul citeste sau seteaza campuri administrative simple. Pentru o versiune ulterioara, designul poate fi intarit prin introducerea unui mutex dedicat pentru toate citirile si scrierile din `SharedState`, mai ales daca numarul de workeri sau tipurile de operatii admin cresc.

12. Build, Analiza si Profilare

Buildul implicit foloseste `-Wall -Wextra -Wpedantic -Werror -std=c11 -D_POSIX_C_SOURCE=200809L`, iar serverul este linkat cu `-pthread` si bibliotecile externe necesare. Makefile construiește executabilele `server`, `client` si `admin`, ruleaza analiza statica prin `make clang-tidy`, expune builduri cu ASan/LSan/UBSan si TSan, ruleaza Valgrind Memcheck si Helgrind si regenereaza documentatia prin `make docs`.

Fisierul `.clang-tidy` activeaza familiile `clang-analyzer`, `bugprone`, `cert`, `concurrency`, `misc`, `performance`, `portability` si `readability`, cu `WarningsAsErrors: ''`. Generatorul local `tools/md_to_pdf.py` transforma sursele Markdown ale documentatiei in fisiere PDF folosind `pdflatex`.

13. Trasabilitate Design - Implementare

Headerul protocolului si codurile de operatii se gasesc in `include/protocol.h`, iar implementarea I/O este in `src/protocol.c`. Structurile de stare si limitele modelului sunt in `include/model.h`, iar parsarea ER, generarea SQL, validarea si rapoartele sunt in `src/model.c`. Configurarea, socketurile, firele de executie, coada, joburile, administrarea si logging-ul sunt in `src/server.c`. Fluxul clientului ordinar C este in `src/client.c`, fluxul admin este in `src/admin_client.c`, interoperabilitatea protocolului este demonstrata in `clients/python_client.py`, iar datele de test sunt in `examples/er_schema.json` si `examples/inserts_*.sql`.

14. Limitari Cunoscute

Formatul ER acceptat este intentionat simplu si urmeaza exemplul din `examples/er_schema.json`. Validatorul semantic acopera forma `INSERT INTO table [(cols...)] VALUES (...), (...);`. Anularea admin este cooperativa, astfel incat pentru operatii foarte scurte efectul poate fi vizibil doar ca cerere in raport. Blocklist-ul este tinut in memorie si nu este persistat intre reporniri. `SharedState` foloseste limite fixe pentru tabele, coloane, randuri, joburi si istoric.